

BANRED

Prerrequisitos de Instalación

Workshop Kong Konnect

Guía simple para preparar la estación de trabajo del alumno

Campo Detalle

Cliente BANRED S.A.

Proveedor PERCEPTIVA SRL

Plataforma Kong Konnect - arquitectura híbrida con Control Plane SaaS y entornos locales de laboratorio

Objetivo Indicar qué debe instalar cada alumno, qué debe estar configurado en Konnect y qué conectividad debe tener desde su estación de trabajo.

Versión 0.4

Fecha 08-may-2026

Importante: no incluir tokens reales de Konnect en documentos, correos, tickets o repositorios. Cada alumno debe usar un PAT propio o un token temporal entregado por el instructor.

1. Objetivo de este documento

Este documento resume, de forma práctica, lo que cada alumno debe tener listo antes del workshop de Kong Konnect. El foco es preparar la laptop para ejecutar ejercicios locales, administrar configuración declarativa con decK, importar colecciones en Insomnia, levantar servicios de laboratorio con Docker y validar conexión contra la instancia de Kong Konnect asignada.

El documento no reemplaza la guía del laboratorio. Su propósito es servir como checklist previo para evitar que el tiempo del workshop se consuma instalando herramientas básicas o resolviendo bloqueos de red.

2. Checklist mínimo antes del workshop

Categoría Requisito Uso en los ejercicios

Sistema operativo macOS, Windows 10/11 o Ejecutar CLI, Docker, Linux con permisos de scripts y pruebas instalación locales.

Docker Docker Desktop o Docker Levantar Kong Data Engine con Docker Plane local, mocks, Compose Keycloak, WireMock, Prism y stack de observabilidad.

Kubernetes local Habilitado en Docker Usado en la preparación Desktop cuando el de entorno local ejercicio lo requiera descrita para los primeros días del workshop.

Git Cliente Git instalado Clonar o recibir el repositorio de assets del workshop.

curl CLI curl disponible Probar APIs, Konnect, mocks y endpoints locales.

jq Procesador JSON de Leer respuestas JSON de línea de comandos APIs y scripts.

decK decK CLI v1.40+ o Sincronizar servicios, versión indicada por el rutas, plugins y instructor consumers contra Konnect.

Insomnia Aplicación desktop Importar colecciones, Insomnia OpenAPI y ejecutar pruebas manuales o Runner.

Terraform Terraform CLI, Gestionar recursos de requerido para plataforma cuando el ejercicios laboratorio lo incluya. APIOps/Portal/Catalog

Node.js / npm / inso Node.js LTS y, si Ejecutar pruebas por aplica, Insomnia CLI terminal con inso y utilitarios de apoyo.

Editor VS Code u otro editor Editar YAML, OpenAPI, de texto .env y scripts.

3. Instalación por sistema operativo

Las siguientes instrucciones son prácticas. Si BANRED utiliza empaquetadores corporativos, Intune, SCCM, Jamf, repositorios internos o restricciones de software, usar el mecanismo corporativo equivalente y luego ejecutar las validaciones de la sección 6.

3.1 macOS

Opción recomendada: Homebrew para herramientas CLI y descarga/instalación de Docker Desktop e Insomnia. Abrir Terminal y ejecutar:

```
# 1) Instalar herramientas base de Apple si no existen
xcode-select --install

# 2) Instalar Homebrew si no está instalado
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# 3) Actualizar Homebrew
brew update

# 4) Instalar herramientas CLI
brew install git curl jq python@3.12 node kong/deck/deck
brew tap hashicorp/tap
brew install hashicorp/tap/terraform

# 5) Instalar aplicaciones desktop
brew install --cask docker insomnia visual-studio-code
```

Después de instalar Docker Desktop:

1. Abrir Docker Desktop desde Applications.
2. Aceptar los términos de uso de Docker Desktop según la política de BANRED.
3. En Settings > Resources asignar al menos 4 GB de memoria si la laptop lo permite.
4. En Settings > Kubernetes habilitar Kubernetes si el instructor confirma que el ejercicio lo requiere.
5. Esperar a que Docker indique estado Running.

Validación rápida en macOS:

```
git --version
curl --version
jq --version
python3 --version
node -v
npm -v
docker version
docker compose version
deck version
terraform version
```

3.2 Windows 10/11

Opción recomendada: Windows 10/11 con WSL 2 y Docker Desktop usando backend WSL 2. Ejecutar Símbolo del Sistema (CMD) como usuario con permisos de instalación.

```
# 1) Instalar WSL 2 con Ubuntu
```

```
wsl --install -d Ubuntu
```

```
wsl --update
```

```
# 2) Reiniciar Windows si el instalador lo solicita
```

Instalar herramientas con winget desde Símbolo del Sistema (CMD):

```
winget install --id Git.Git -e
```

```
winget install --id Docker.DockerDesktop -e
```

```
winget install --id Kong.Insomnia -e
```

```
winget install --id Hashicorp.Terraform -e
```

```
winget install --id OpenJS.NodeJS.LTS -e
```

```
winget install --id Microsoft.VisualStudioCode -e
```

```
# jq puede instalarse con winget si está disponible en el catálogo corporativo:
```

```
winget install --id jqlang.jq -e
```

Instalar deck en Windows. Usar la versión indicada por el instructor; el ejemplo usa una variable para facilitar actualización:

```
$DECK_VERSION = "1.59.1"
```

```
New-Item -ItemType Directory -Force "$HOME\bin" | Out-Null
```

```
Invoke-WebRequest `
```

```
"https://github.com/Kong/deck/releases/download/v1.59.1/deck-1.59.1-windows-amd64.tar.gz"
```

```
,
```

```
-OutFile "$HOME\Downloads\deck.tar.gz"
```

```
tar -xzf "$HOME\Downloads\deck.tar.gz" -C "$HOME\bin"
```

```
[Environment]::SetEnvironmentVariable(
```

```
"Path",
```

```
[Environment]::GetEnvironmentVariable("Path", "User") + ";$HOME\bin",
```

```
"User"
```

```
)
```

```
# Cerrar y abrir una nueva terminal antes de validar deck.
```

Configuración de Docker Desktop en Windows:

6. Abrir Docker Desktop.

7. Verificar que se usa WSL 2 como backend.

8. En Settings > Resources > WSL Integration habilitar integración con Ubuntu.

9. En Settings > Kubernetes habilitar Kubernetes si el instructor confirma que el ejercicio lo requiere.

10. Usar Linux containers, no Windows containers.

Validación rápida en Windows CMD:

```
git --version
```

```
curl.exe --version
```

```
jq --version
```

```
node -v
```

```
npm -v
```

```
docker version
```

```
docker compose version
```

```
deck version
```

```
terraform version
```

```
wsl --status
```

Para ejecutar scripts bash del laboratorio, usar Ubuntu en WSL o Git Bash. La ruta recomendada para repositorios dentro de WSL es el filesystem Linux, por ejemplo:

```
mkdir -p ~/workshop
cd ~/workshop
```

3.3 Linux

Las instrucciones siguientes cubren Ubuntu/Debian. Para Fedora/RHEL, usar dnf y paquetes rpm equivalentes. En Linux se puede usar Docker Desktop o Docker Engine; para el workshop, Docker Engine con Docker Compose suele ser suficiente, salvo que se requiera Kubernetes local.

3.3.1 Ubuntu/Debian - herramientas base

```
sudo apt-get update
sudo apt-get install -y ca-certificates curl gnupg lsb-release git jq unzip python3 python3-pip
```

3.3.2 Ubuntu/Debian - Docker Engine y Docker Compose

```
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo
${UBUNTU_CODENAME:-$VERSION_CODENAME}) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
sudo usermod -aG docker $USER

# Cerrar sesión y volver a entrar para que aplique el grupo docker.
```

3.3.3 Ubuntu/Debian - Terraform

```
wget -O- https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o /usr/share/keyrings/hashicorp-archive-
keyring.gpg

echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://apt.releases.hashicorp.com
$(lsb_release -cs) main" | sudo tee /etc/apt/sources.list.d/hashicorp.list

sudo apt-get update
sudo apt-get install -y terraform
```

3.3.4 Ubuntu/Debian - Node.js LTS

```
curl -fsSL https://deb.nodesource.com/setup\_lts.x | sudo -E bash -
sudo apt-get install -y nodejs
node -v
npm -v
```

3.3.5 Ubuntu/Debian - deck

```
DECK_VERSION="1.59.1"
curl -sL
https://github.com/Kong/deck/releases/download/v\${DECK\_VERSION}/deck\_\${DECK\_VERSION}\_linux\_amd64.tar.gz
-o deck.tar.gz
tar -xzf deck.tar.gz
sudo mv deck /usr/local/bin/deck
rm deck.tar.gz

deck version
```

3.3.6 Ubuntu/Debian - Insomnia

Descargar el paquete Linux desde el sitio oficial de Insomnia. Si se descarga un .deb, instalarlo así desde la carpeta Downloads:

```
cd ~/Downloads
sudo apt install ./Insomnia*.deb
```

Validación rápida en Linux:

```
git --version
curl --version
jq --version
python3 --version
node -v
npm -v
docker version
docker compose version
deck version
terraform version
```

4. Configuración requerida en Kong Konnect

Cada alumno debe tener acceso a la organización de Konnect usada en el workshop y al Control Plane asignado para los ejercicios. El instructor o administrador de la plataforma debe validar estos puntos antes de iniciar.

Elemento Debe estar listo Validación esperada

Cuenta Konnect Usuario invitado y con El alumno puede iniciar acceso a la sesión en Konnect. organización del workshop.

Control Plane Control Plane El alumno puede ver el disponible para los Control Plane en ejercicios. Para esta Konnect. guía: Local AI Demo Gateway.

Permisos Permisos suficientes El alumno puede para leer Data Plane ejecutar deck gateway Nodes y administrar ping y sincronizar servicios, rutas, configuración si el plugins, consumers y ejercicio lo requiere. credenciales según el laboratorio.

Personal Access Token PAT personal o token El token permite temporal provisto por conexión de deckK a el instructor. No debe Konnect. compartirse ni documentarse.

Data Plane local Assets, certificados o En Konnect, el Data docker-compose Plane debe aparecer provistos por el conectado y, cuando instructor si el alumno aplique, en estado In debe levantar su propio Sync. Data Plane.

4.1 Variables de entorno

Usar los valores definidos para el workshop. El token real debe ser introducido por cada alumno en su propia estación de trabajo.

```
# macOS / Linux / WSL
export KONNECT_TOKEN="<PAT_PERSONAL_O_TOKEN_TEMPORAL>"
export KONNECT_ADDR="https://us.api.konghq.com/" export CP_ID="CP_ID_PERSONAL"
export CP_NAME="Local AI Demo Gateway"
export CONTROL_PLANE_NAME="$CP_NAME"
```

```
# Windows CMD - solo sesión actual
set KONNECT_TOKEN=<PAT_PERSONAL_O_TOKEN_TEMPORAL>
set KONNECT_ADDR=https://us.api.konghq.com/ set CP_ID=CP_ID_PERSONAL
set CP_NAME=Local AI Demo Gateway
$env:CONTROL_PLANE_NAME=%CP_NAME%

# Windows CMD - persistente para el usuario
setx KONNECT_ADDR "https://us.api.konghq.com/" setx CP_ID "CP_ID_PERSONAL"
setx CP_NAME "Local AI Demo Gateway"
setx CONTROL_PLANE_NAME "Local AI Demo Gateway"
# No usar setx para tokens si la política de seguridad de BANRED lo prohíbe.
```

4.2 Archivo .deck.yaml

Si el repositorio del workshop incluye un script de setup, usar ese script. Si se requiere crear el archivo manualmente, desde la carpeta del laboratorio:

```
# macOS / Linux / WSL
cat > .deck.yaml <<EOF
konnnect-token: "$KONNECT_TOKEN"
konnnect-addr: "$KONNECT_ADDR"
konnnect-control-plane-name: "$CP_NAME"
EOF

# Validar
deck gateway ping

# Windows CMD
echo konnect-token: %KONNECT_TOKEN% > .deck.yaml
echo konnect-addr: %KONNECT_ADDR% >> .deck.yaml
echo konnect-control-plane-name: %CP_NAME% >> .deck.yaml

# Validar
deck gateway ping
```

5. Conectividad requerida desde la estación de trabajo

Antes del workshop, validar que la red corporativa, VPN, proxy, antivirus y firewall local no bloqueen la comunicación necesaria. Si BANRED usa inspección TLS, puede ser necesario permitir excepciones para los dominios de Konnect y repositorios de software.

Destino / Puerto Uso Requisito

<https://us.api.konghq.com:443> API de Konnect usada Salida HTTPS permitida por deck y scripts. desde la laptop.

Dominios runtime de Konnect / Conexión del Data Plane Salida TLS permitida; *.konghq.com:443 local con el Control evitar ruptura por Plane SaaS. inspección TLS.

github.com / Descarga de deck, Salida HTTPS permitida. releases.githubusercontent.com:443 assets y dependencias.

registry-1.docker.io / Descarga de imágenes Salida HTTPS permitida; auth.docker.io / Docker Hub:443 Docker del laboratorio. autenticarse si la red tiene rate limits.

npmjs.org / nodejs.org / Node.js, npm e inso CLI Salida HTTPS permitida. deb.nodesource.com:443 cuando aplique.

releases.hashicorp.com / Instalación de Salida HTTPS permitida. apt.releases.hashicorp.com:443 Terraform.

insomnia.rest / GitHub releases:443 Descarga de Insomnia. Salida HTTPS

permitida.

5.1 Puertos locales que deben estar libres

Los ejercicios pueden levantar varios contenedores locales. Antes de iniciar, cerrar servicios que estén usando estos puertos o solicitar al instructor variables alternativas.

Puerto local	Servicio típico	Uso en laboratorio
--------------	-----------------	--------------------

8000	Kong Proxy	Entrada principal para probar APIs expuestas por Kong.
------	------------	--

8010	Segundo Data Plane	Ejercicio de clustering opcional / escalabilidad.
------	--------------------	---

8080	Prism Mock	Mock de API basado en OpenAPI.
------	------------	--------------------------------

8081	httpbin	Backend echo para validar headers, body y transformaciones.
------	---------	---

8082	WireMock	Mock de APIs de negocio.
------	----------	--------------------------

8083	Keycloak	Servidor OIDC local para ejercicios de autenticación.
------	----------	---

8100	Kong metrics	Endpoint local de métricas Prometheus.
------	--------------	--

3000	Grafana	Dashboards de observabilidad local.
------	---------	-------------------------------------

16686	Jaeger UI	Visualización de trazas.
-------	-----------	--------------------------

9090	Prometheus	Métricas.
------	------------	-----------

3100	Loki	Logs.
------	------	-------

4317 / 4318 OpenTelemetry Recepción de trazas OTLP gRPC / HTTP.

6. Validación antes del workshop

Ejecutar esta validación al menos un día antes del workshop. Si falla cualquier punto, levantarlo como bloqueo operativo.

6.1 Validar herramientas locales

```
git --version
curl --version
jq --version
python3 --version || python --version
node -v
npm -v
docker version
docker compose version
deck version
terraform version
```

6.2 Validar Docker

```
docker run --rm hello-world
docker compose version
```

6.3 Validar conectividad con Konnect

```
curl -I https://us.api.konghq.com/
deck gateway ping
```

6.4 Validar Data Plane local cuando esté levantado

```
curl -i http://localhost:8000/qualquer-rota # Respuesta esperada si Kong está activo pero sin rutas:  
# HTTP/1.1 404 Not Found  
# {"message":"no Route matched with those values"}
```

7. Problemas comunes y corrección rápida

Síntoma Causa probable Acción sugerida

docker: command not found Docker no instalado o Abrir nueva terminal. found PATH no actualizado. En Windows verificar Docker Desktop + WSL Integration.

Cannot connect to the Docker Desktop no está Iniciado Docker Desktop. Docker daemon iniciado o el usuario En Linux ejecutar no pertenece al grupo usermod -aG docker docker en Linux. \$USER y reabrir sesión.

deck gateway ping falla PAT inválido, expirado Regenerar PAT o validar con 401/403 o sin permisos. permisos en Konnect.

deck gateway ping no Proxy, VPN, DNS o Validar salida HTTPS a resuelve DNS o timeout firewall bloquean us.api.konghq.com y salida a Konnect. dominios runtime de Konnect.

Puerto local ocupado Otro proceso usa el Identificar proceso y puerto requerido. liberar puerto, o pedir al instructor variable alternativa.

Docker no descarga Bloqueo a Docker Hub, Configurar proxy imágenes proxy no configurado o corporativo en Docker rate limit. Desktop o usar mirror corporativo.

Insomnia no importa Archivo incorrecto o Actualizar Insomnia y colección versión desactualizada. volver a importar el workspace entregado.

8. Referencias oficiales para instalación

Estas referencias se incluyen para alumnos que necesiten validar compatibilidad, versiones actuales o instrucciones alternativas de instalación.

- Docker Desktop: <https://docs.docker.com/desktop/>
- Docker Desktop para macOS: <https://docs.docker.com/desktop/setup/install/mac-install/>
- Docker Desktop para Windows: <https://docs.docker.com/desktop/setup/install/windows-install/>
- Docker Desktop para Linux: <https://docs.docker.com/desktop/setup/install/linux/>
- Kong deck: <https://developer.konghq.com/deck/>
- Insomnia: <https://insomnia.rest/>
- Terraform: <https://developer.hashicorp.com/terraform/install>
- Node.js / npm: <https://docs.npmjs.com/downloading-and-installing-node-js-and-npm/>
- jq: <https://jqlang.org/>